

Introduction to Quantum Algorithms.

①

1. What is Quantum Algorithm?
2. Why we need Quantum Algorithm?
3. How can we use/design Quantum Algorithm?

Now, we answer the above questions one by one.

First, we need know quantum computer. While in classical computing, bits can represent either a "0" or a "1", quantum bits or qubits can exist in superpositions of these values.

Classical $\Rightarrow$ quantum Schrödinger's cats
 $\{0, 1\} \Rightarrow [0, 1]$

Second, quantum register with several qubits to store and process exponentially more information than classical register with an equivalent number of bits. However, harnessing this property of quantum computers presents significant challenges.

Peter Shor: quantum polynomial-time factoring and discrete log algorithm.

public-key cryptography $\xRightarrow{\text{vulnerability}}$ quantum computer.

Lo Grover: quadratic speedup for unstructured search problem.
 $O(N) \Rightarrow O(\sqrt{N})$

HHL: properties of solutions of large sparse linear system (exp speedup).

Classical $\xRightarrow{\text{speedup}}$ quantum
(big data, machine learning).

Third, how to use / design. Quantum Algorithm need all this book. In chapter I, I will introduce the following three topics:

1.1) Deterministic $\Leftrightarrow$ Probability
 $P \Leftrightarrow NP$

1.2) Boolean $\Leftrightarrow$ Circuit.

1.3) Reversible Circuits.

1.1. Turing machine model (1963).

* Def. 1.1.1. An alphabet is a finite nonempty set. ($\mathbb{Q}, \mathbb{Z}, \mathbb{R}, \mathbb{N}$).

* Logic operation:

Table 1.1.3. Permitted Logic operations.

important!

Name	Logic operator	ab	$a \wedge b$	$a \vee b$	$\neg a$	$a \uparrow b$	$a \downarrow b$	$a \oplus b$
AND	$\wedge$	00	0	0	1	1	1	0
OR	$\vee$	01	0	1	1	1	0	1
NOT	$\neg$	10	0	1	0	1	0	1
NAND	$\uparrow$	11	1	1	0	0	0	0
NOR	$\downarrow$							
XOR	$\oplus$							

* Deterministic Algorithms.

Algorithm 1.1.6 Euclidean algorithm.

Input $a, b \in \mathbb{Z}$

Output $\text{gcd}(a, b)$

1. $\text{gcd}(a, b)$ (Greatest Common Divisor)
2. $a \leftarrow |a|$
3. $b \leftarrow |b|$

4. while $b \neq 0$ do
 $r \leftarrow a \bmod b$
 $a \leftarrow b$
 $b \leftarrow r$
end while. return a end

* Time and space complexity.

Time complexity:
 running time of $O(\text{size}(a))$.

$\text{Size}(a) \xrightarrow{\text{time}} \text{output}$

Space complexity

$\text{Size}(a) \xrightarrow{\text{space}} \text{input}$
memory space

* Probabilistic Algorithms.

- (1) Monte Carlo algorithms: They always terminate but may not always be successful.
- (2) Las Vegas algorithms: They may not terminate but when they terminate, they are successful.

Algorithm 1.2.2. (Monte Carlo)

Input $k \in \mathbb{N}$

Output $s \in \{0,1\}^k$

1. Random String C_k
2. for $i=0$ to $k-1$ do
- 3 $s_i \leftarrow \text{CoinToss}$
4. end for
5. return $s = (s_0, \dots, s_{k-1})$
6. end

Algorithm 1.2.6 (Las Vegas factoring).

Input. $a \in \mathbb{N}, a > 1$

Output. A proper divisor $b \in \mathbb{N}$ of a .

1. $\text{IVFactor}(a)$
2. repeat
3. $b \leftarrow \text{mcFactor}(a)$
4. until $b \neq \text{"Failure"}$
5. return b .
6. end.

* Analysis of probabilistic algorithms.

Let $k \in \mathbb{N}_0$, $k \leq l$, and let $\vec{r} = (r_0, \dots, r_{k-1})$ be a prefix of a random sequence of a run of A with input a . Also, for $0 \leq i < k$ denote by p_i the probability for the return value r_i to occur. Then we see

$$\Pr_{A,a}(\vec{r}) = \prod_{i=0}^{k-1} p_i \quad \vec{r} \in \{0,1\}^k \quad \Pr_{A,a}(F) = \frac{1}{2^k}$$

We denote the set of all random sequences of runs of A with input a by $\text{Rand}(A,a)$ and the set of finite string in $\text{Rand}(A,a)$ by $\text{FRand}(A,a)$.

Lemma 1.3.2. Let $a \in \text{Input}(A)$. The sum

$$\sum_{\vec{r} \in \text{FRand}(A,a)} \Pr_{A,a}(\vec{r})$$

converges in $[0,1]$. $\Pr_{A,a}(\infty) = 1 - \sum_{\vec{r} \in \text{FRand}(A,a)} \Pr_{A,a}(\vec{r}) = 0 \Rightarrow$

$(\text{FRand}(A,a), \Pr_{A,a})$ is a discrete probability space.

P: polynomial time algorithm (deterministic).

NP: give a answer, can be verified in polynomial.

Example: (174) Goldbach conjecture: Every even integer ≥ 4 is the sum of two odd prime numbers.

$$4 = 1 + 3 \quad 6 = 3 + 3 \quad 8 = 3 + 5$$

We give a; and prime number p . If $a-p$ is also a prime number.

However, we can not prove Goldbach conjecture in a polynomial time.

(1 million \$) $\Leftarrow$ (P $\Leftrightarrow$ NP) seven millennium prize problem.

1.2. Boolean $\Leftrightarrow$ Circuit.

Table 1.5.1

Name	Logic	Circuit.
AND	$\wedge$	$\odot$
OR	$\vee$	$\rightarrow$
NOT	$\neg$	$\neg$
NAND	$\uparrow$	$\odot$
NOR	$\downarrow$	$\rightarrow$
XOR	$\oplus$	$\odot$

Theorem 1.5.7. The set {NOT, AND, OR} is universal for classical computation.

$$\begin{cases} \text{NOR} = \text{NOT}(\text{OR}) \\ \text{NAND} = \text{NOT}(\text{AND}) \\ \text{XOR} = [\text{OR}] \text{ AND } (\text{NOT}(\text{AND})) \end{cases}$$

Theorem 1.5.9 The gate set {NAND} is universal for classical computation.

$$\text{NOT} = \text{NAND}(b, b) \quad \text{AND}(a, b) = \text{NAND}(\text{NAND}(a, b), \text{NAND}(a, b))$$

$$\text{NAND}(\text{NAND}(b, b), \text{NAND}(a, a)) = \text{OR}(a, b)$$

$\Rightarrow$ {NOR} is universal for classical computation.

$$\text{NOR}(\text{NOR}(a, b), \text{NOR}(a, b)) = \text{NAND}$$

Definition 1.5. Let G be a Universal set of logic gates and let f be a Boolean function. The circuit size complexity of f with respect to G is the minimum size of any circuit that compute f .

Theorem 1.6.6 For all function $f: \{0,1\}^* \rightarrow \{0,1\}^*$, computation of problems CP , and language L , there is a circuit family that computes f , solves CP , or decides L .

Boolean $\Leftrightarrow$ circuit.

Boolean $\Leftrightarrow$ quantum circuit.

* 1.3. Reversible Circuits.

Section 4.7

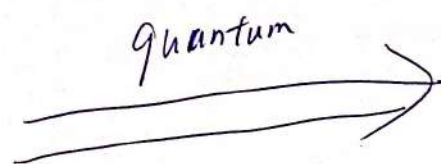
CNOT

SWAP

CCNOT

FANOUT

Fredkin (CCSWAP)



CNOT

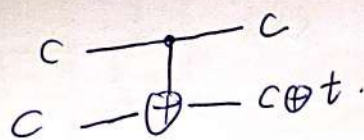
SWAP

CCNOT

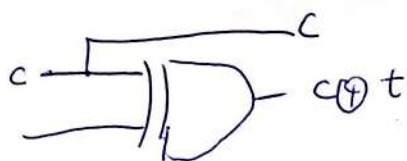
FANOUT

Fredkin

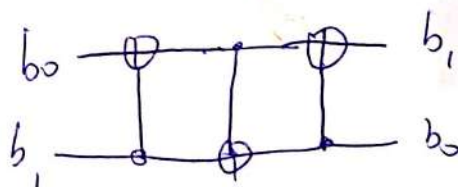
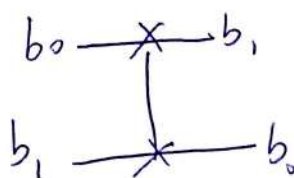
CNOT



NOT operation under control c.



SWAP

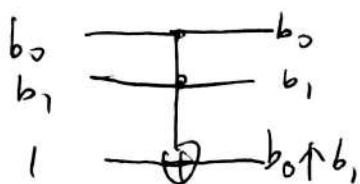


Proposition 1.7.2. Let $n \in \mathbb{N}$ and let $\pi \in S_n$. Then the map.

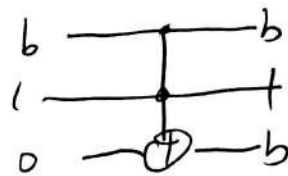
Proof: (1.7.1) $f_\pi: \{0,1\}^n \rightarrow \{0,1\}^n$, $(b_0, \dots, b_{n-1}) \mapsto (b_{\pi(0)}, \dots, b_{\pi(n-1)})$
Each permutation π is the product of at most $n-1$ transpositions.

Toffoli or CCNOT: $\{0,1\}^3 \rightarrow \{0,1\}^3$ $(c, b, a) \mapsto (c, b, c \wedge b \wedge a)$ (6)

Use CCNOT to derive NAND and FANOUT gates.



NAND

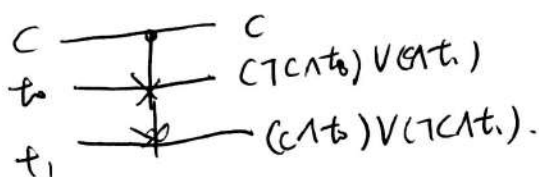


FANOUT

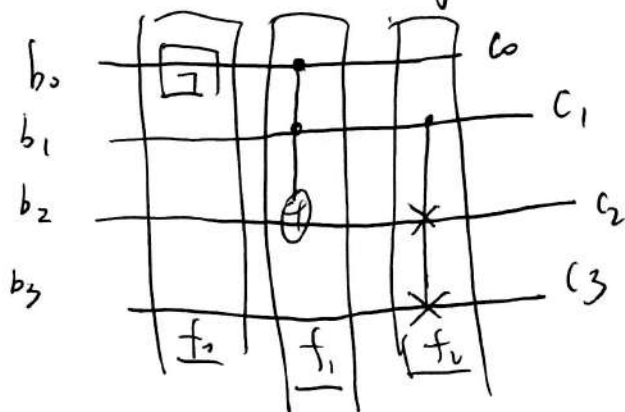
$$(b \wedge 1) \oplus 0 = b \oplus 0 = (b \wedge 1) \vee (1 \wedge 0) = b \vee 0 = b$$

$$((b_0 \wedge b_1) \vee 1) \wedge \neg((b_0 \wedge b_1) \wedge 1) = 1 \wedge \neg(b_0 \wedge b_1) = \neg(b_0 \wedge b_1) = b_0 \uparrow b_1$$

Frederick (CSWAP): $\{0,1\}^3 \rightarrow \{0,1\}^3$ $(c, b, a) \mapsto (c, \neg c \wedge b \vee a, (c \wedge b) \vee \neg a)$



1.7.2 construction of reversible circuits.



$$f = f_2 \circ f_1 \circ f_0$$

$$f_0 = (\text{NOT}, 1, 1)$$

$$f_1 = (\text{CCNOT}, 1, 1)$$

$$f_2 = (\neg, \text{CSWAP})$$

$$(0, 1, 0, 0) \xrightarrow{f_0} (1, 1, 0, 0) \xrightarrow{f_1} (1, 1, 1, 0) \xrightarrow{f_2} (1, 1, 0, 1)$$

1.7.3. f may not be reversible.

find h reversible s.t. $h(y) = f(x)$, $\forall x \in \{0,1\}^n$.

Theorem 1.7.6. For all Boolean function f , there exist.

a reversible Cr (use CCNOT), $g: \{0,1\}^n \rightarrow \{0,1\}^{n+p-m}$, $(p \leq 2|S|_F)$

$$(1.7.7) \quad h: \{0,1\}^n \times \{0,1\}^p \rightarrow \{0,1\}^m \times \{0,1\}^{n+p-m}$$

$$(1.7.8) \quad h(x, a) = (f(x), g(x))$$

ancilla bits

garbage

$|f|_F = 0$. $p=0$ $C_r = C$ $\vec{a} = ()$. $g: \{a, b\}^n \rightarrow \{a, b\}^m$, $\vec{x} \rightarrow ()$.

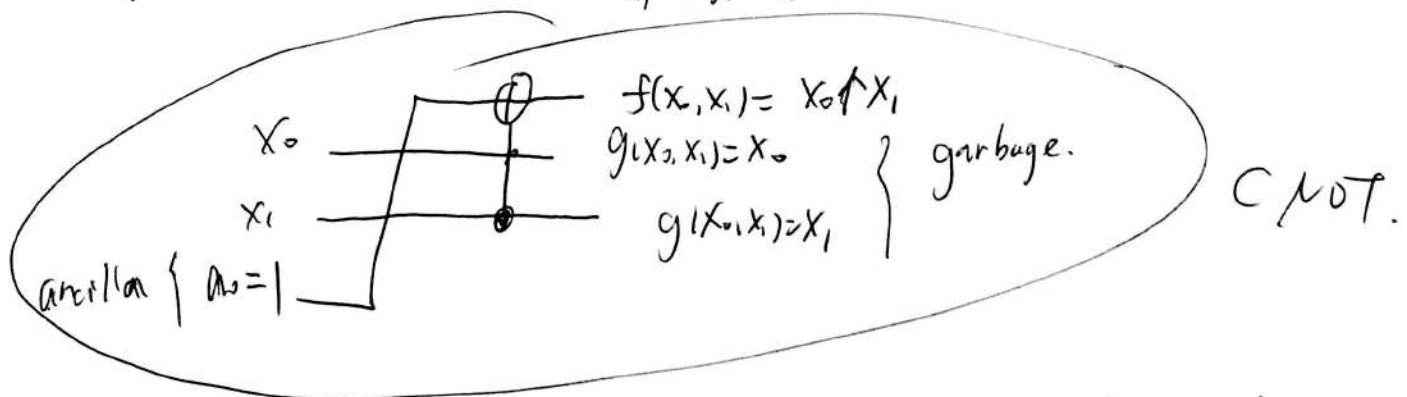
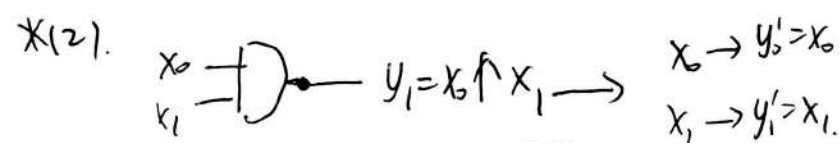
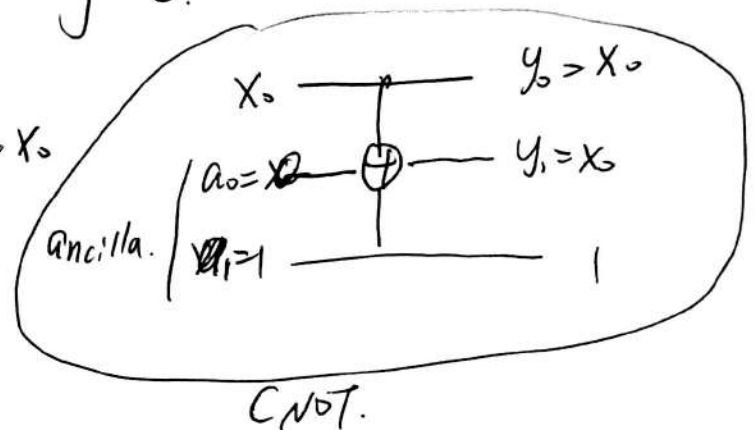
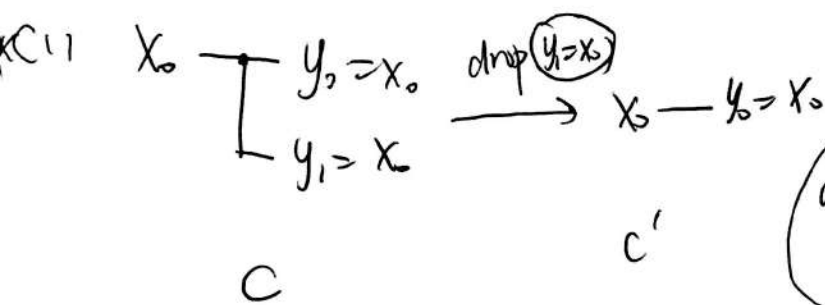
$|f|_F > 0$. $f: \{0, 1\}^n \rightarrow \{0, 1\}^m$. Let f' $|f'|_F < K$. The 1.7.10 holds

Let C be a circuit that implements f , use only NAND and FANOUT gates. $|f|_F = K$.

Then C has at least one of the following properties.

(1). $\exists$ FANOUT in C whose outgoing edges are incoming edges of two output nodes y_i and y_j of C .

(2). $\exists$ NAND in C whose outgoing edges is the incoming edge of an output node y_i of C .



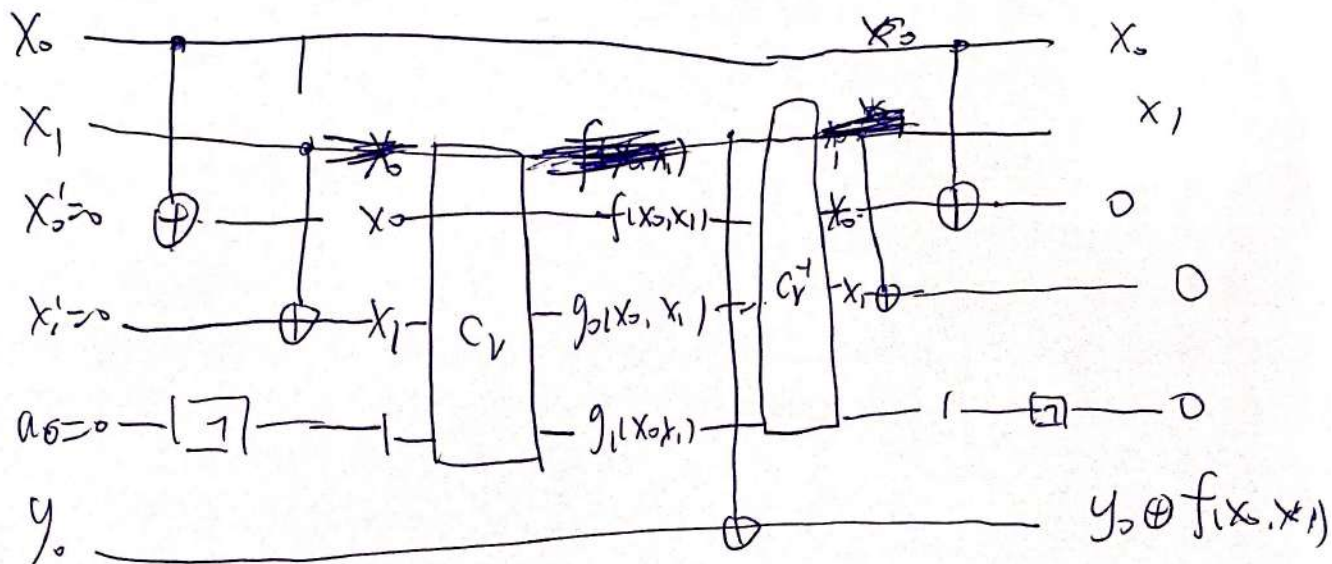
When we use construction from Th. 1.7.10., the garbage may be problematic.

uncompute trick

Theorem 1.7.12. For all Boolean function $f: \{0,1\}^n \rightarrow \{0,1\}^m$, there is $p \in \mathbb{N}_0$, $p \leq 2|f|_F$, a reversible P_k with $|P_k| = O(|f|_F)$ that use only Toffoli, NOT, and CNOT, such that P_k implements a function

$$(1.7.9) \quad h: \{0,1\}^n \times \{0,1\}^{n+p} \times \{0,1\}^m \rightarrow \{0,1\}^n \times \{0,1\}^{n+p} \times \{0,1\}^m$$

with $h(x, 0, y) = (x, 0, y \oplus f(x))$.



Note that $g \rightarrow 0$ garbage (uncompute trick)
 $g_1 \rightarrow 0$

Exercise. $f(b_0, b_1) = b_0 \downarrow b_1$.

